

SQ26 — ISIC Rev.5 Classification

of QDArchive Research Dataset Projects

Methodology Report · Part 2

SQ26 — ISIC Rev.5 Classification of QDArchive Projects

Methodology Report · Part 2

Student ID: 23692652
University: FAU Erlangen-Nürnberg
Supervisor: Prof. Dirk Riehle
Date: July 2026

1. Overview

This document describes the complete, step-by-step methodology used to classify 53,080 research dataset projects from three open-data repositories — Zenodo (repo 1), Harvard Dataverse (repo 10), and Columbia University Libraries (repo 19) — according to the ISIC Rev.5 standard (International Standard Industrial Classification of All Economic Activities, Revision 5, United Nations 2025).

Each project received:

- A primary class at the ISIC section level (one of 22 letters, A-V)
- A secondary class at the ISIC division level (one of 88 two-digit codes)
- A confidence score derived from classifier agreement

The classification used a four-classifier voting ensemble. No single algorithm was relied upon; each classifier provided an independent signal, and their votes were combined to produce a robust final decision.

2. Data Source and Project Types

2.1 Repositories

Repository ID	Name	Projects
1	Zenodo	39,405
10	Harvard Dataverse	13,650
19	Columbia University Libraries	25
Total		53,080

2.2 Project Types

Projects in the database are typed into four categories based on their file contents:

Type	Description	Count
QD_PROJECT	Contains qualitative data files.	35,289
OTHER_PROJECT	Research datasets of other kinds.	15,915
QDA_PROJECT	Contains QDA software projects.	67.
NOT_A_PROJECT	Entries that do not represent a project.	1,809

2.3 Input Features per Project

Each project record in the SQLite database (23692652-sq26.db) contains:

- title — the dataset title as deposited by the author
- description — a free-text abstract or description
- keywords — zero or more author-supplied keywords
- files — list of filenames deposited (e.g., interview_transcripts.xlsx)

3. Classification Target: ISIC Rev.5

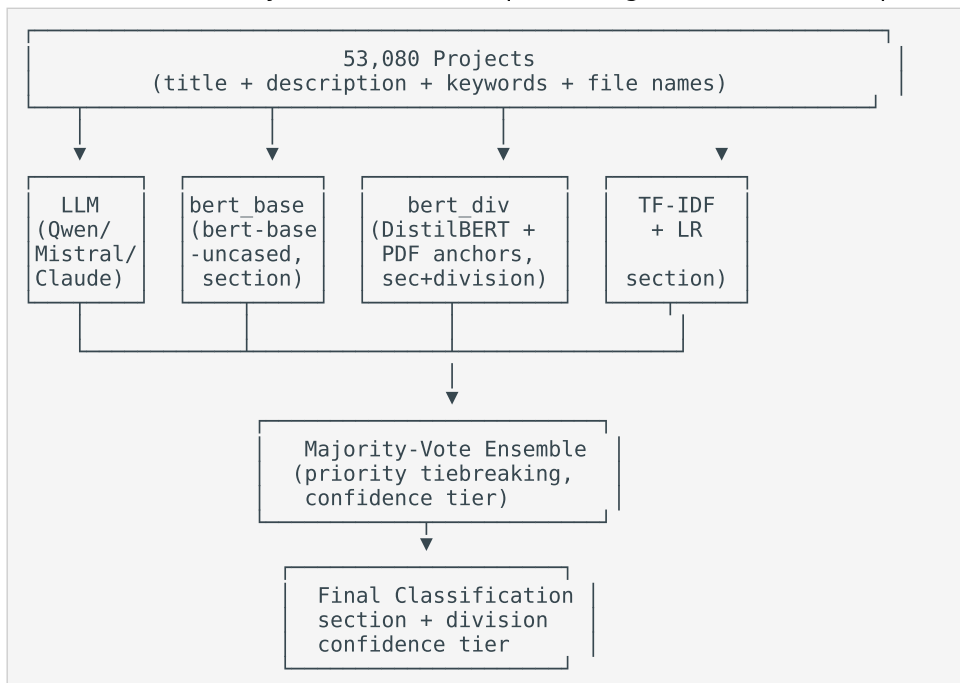
ISIC Rev.5 is the United Nations standard for classifying economic activities. It has a two-level hierarchy:

- 22 Sections (single letter A-V): broad economic sectors (e.g., A = Agriculture, N = Professional/Scientific activities)
- 88 Divisions (two-digit code): more specific sub-categories within each section (e.g., division 72 = Scientific research and development within section N)

The classification principle applied was: classify by subject matter — what the dataset is *about*, not the fact that it is a scientific research dataset. This is consistent with the ISIC Rev.5 Explanatory Notes, which state that research *about* a topic belongs in the relevant subject-matter section (e.g., a survey on voting belongs in P/84, not N/72), while only datasets that are themselves products of the science/technology industry belong in N/72.

4. Classification Pipeline

The pipeline consists of four independent classifiers followed by a voting ensemble. Each classifier writes its results to a dedicated table in the SQLite database; the tables are never overwritten by other classifiers, preserving each model's independent judgment.



Step 1 — LLM Classifier (`classify_combined.py`, `classify_isic.py`)

Table: `classifications_combined`

Models used: Qwen2.5:7b (19,019 projects), Mistral (1,000), Claude Haiku 4.5 via Batch API (38,383 projects)

This is the highest-quality and highest-priority classifier. It sends each project's

metadata to a large language model with a structured prompt asking for ISIC section and division.

Algorithm:

1. For each project, assemble a prompt containing: title, description, keywords, and file names.
2. The prompt includes the full ISIC Rev.5 section and division list and instructs the model to pick the best match by subject matter.
3. The model returns a JSON object: {"section": "R", "division": "86", "confidence": "high"}.
4. Results are stored in `classifications_combined` with the model name recorded.

LLM deduplication: Projects classified by multiple models are deduplicated by priority — qwen2.5:7b > mistral > claude-haiku-4-5. Only the highest-priority model's result is used in voting.

Output columns: section, section_name, division, division_name, confidence, model_name

Division coverage: qwen2.5:7b and mistral provide full section + division (20,019 projects). Claude Haiku provides section only (38,383 projects).

Section agreement: reference source — highest accuracy.

Step 2 — Training Label Creation (`create_training_labels.py`)

Table: `labels_training`

Labels created: ~50,000

Because classifiers 3 and 4 require supervised training data, this step constructs a high-quality training set from the LLM output. For each project where the LLM assigned a classification, that label is used as a "silver label" (trusted training label). Class balancing is applied to prevent the dominant section (N, ~25%) from overwhelming minority sections.

Step 3 — bert-base-uncased Fine-tuned Classifier (`classify_bert.py`)

Table: `classifications_bert_base`

Model: bert-base-uncased (110M parameters, fine-tuned)

A transformer-based text classifier fine-tuned on the silver training labels. It learns which vocabulary patterns correspond to each ISIC section.

Algorithm:

1. Training phase: Load training labels from `labels_training`. Concatenate title + description + keywords + file names per project (truncated to 512 tokens). Fine-tune bert-base-uncased with a classification head over 22 output classes for 3 epochs using weighted cross-entropy loss. Device: Apple Silicon MPS > CUDA > CPU.
2. Inference phase: Run all 53,080 projects through the fine-tuned model. The softmax output gives probability distributions over all 22 sections; the top-1 is the predicted section. Confidence derived from top-1 softmax probability (≥ 0.80 = very_high, ≥ 0.60 = high, ≥ 0.40 = medium, else low).
3. Results written to `classifications_bert_base` — section-level only.

Section agreement with qwen: 74.0% (on overlapping 19,019 projects)

Section agreement with claude: 77.6% (on overlapping 38,383 projects)

Confidence distribution: very_high 69.2%, high 16.2%, medium 11.0%, low 3.6%

Strengths: Captures long-range semantic dependencies; strongest trained classifier for section; generalises well to rare sections.

Limitation: Section-level output only (no division column).

Step 4 — Division BERT Classifier (`classify_bert_division.py`)

Table: `classifications_bert_division`

Model: `distilbert-base-uncased` (fine-tuned for section + division simultaneously)

A transformer model fine-tuned to produce both section and division labels. It is trained on LLM silver labels augmented with ISIC Rev.5 PDF descriptions as synthetic training examples, making it the only trained classifier capable of predicting all 88 ISIC divisions.

Algorithm:

1. Training data: LLM division labels from `qwen2.5:7b` and `mistral` (20,019 projects with division). Augmented with ISIC Rev.5 Explanatory Notes — for each of the 88 divisions, the "This division includes..." and "This division excludes..." paragraphs are extracted from the official PDF and used as additional training examples.
 2. Training phase: Fine-tune DistilBERT with dual classification heads — one over 22 sections, one over 88 divisions. Joint training with weighted cross-entropy loss. 3 epochs, device: MPS > CUDA > CPU.
 3. Inference phase: Run all 53,080 projects. Predict both section (top-1 of 22) and division (top-1 of 88) simultaneously.
 4. Results written to `classifications_bert_division` — section AND division, 100% coverage.
- Section agreement with `qwen`: 75.1%
Division agreement with `qwen`: 71.5%
Confidence distribution: `very_high` 27.2%, `high` 22.6%, `medium` 21.9%, `low` 28.3%
Strengths: Full division coverage for all 53,080 projects; trained on ISIC PDF descriptions for domain knowledge; essential for division labels where LLM gave section-only output.
Limitation: Trained primarily on `qwen/mistral` labels, so section predictions diverge from `claude-haiku` (40.9% agreement with `claude` vs 75.1% with `qwen`).
-

Step 5 — TF-IDF + Logistic Regression (`classify_tfidf.py`)

Table: `classifications_tfidf`

Model: `sklearn.TfidfVectorizer` + `LogisticRegression`

A lightweight, fast classical machine-learning classifier trained on the same silver labels as the BERT classifiers.

Algorithm:

1. Feature extraction: Build a TF-IDF matrix from project title + description + keywords + file names. Parameters: `sublinear_tf=True`, bigrams (`ngram_range=(1,2)`), maximum 20,000 features, English stop words removed.
 2. Training: Fit multinomial Logistic Regression with `class_weight='balanced'` and L2 regularisation.
 3. Inference: Apply the fitted pipeline to all 53,080 projects. Top-1 predicted probability becomes the confidence score.
 4. Results written to `classifications_tfidf` — section-level only.
- Section agreement with `qwen`: 70.1%
Section agreement with `claude`: 69.6%
Confidence distribution: `very_high` 16.9%, `high` 22.1%, `medium` 22.9%, `low` 38.0%
Strengths: Extremely fast (seconds for 53k projects); interpretable bag-of-words approach;
-

orthogonal to neural classifiers.

Limitation: Cannot capture word order or semantic meaning beyond bigrams; no division output; 38% of projects receive low confidence.

5. Voting Ensemble (`classify\_vote.py`)

After all four classifiers have run independently, their results are combined into a single final classification for each project through majority voting with priority-order tiebreaking.

5.1 Classifier Priority Order

When classifiers disagree, the following priority order determines the tiebreaker (index 1 = highest priority):

Priority	Source	Table	Division?
1	ensemble (LLM)	classifications_combined	Yes (qwen/mistral only)
2	bert_base	classifications_bert_base	No
3	bert_div	classifications_bert_div	Yes (100%)
4	tfidf	classifications_tfidf	No

5.2 Vote Resolution Algorithm

For each of the 53,080 projects:

1. Collect votes: for each of the 4 classifiers, fetch the predicted section.
→ `project_votes = {source: {section, section_name, division, division_name}}`
2. Count section votes:
`section_counts = Counter(section for all votes)`
`max_votes = maximum count`
3. Find winning section:
If only one section has `max_votes` → it wins.
If multiple sections are tied:
Walk PRIORITY order; first classifier in that list whose vote is in the tied set determines the winner.
4. Assign confidence tier:
`very_high` → 4 classifiers agree on winning section
`high` → exactly 3 agree
`medium` → exactly 2 agree
`low` → all classifiers disagree (single-source LLM fallback)
5. Determine section name and division:
Walk PRIORITY order through classifiers that voted for the winning section.
Collect `section_name` from the first agreeing source.
Independently collect `division` + `division_name` from the first agreeing source that has a non-empty division (`bert_base` and `tfidf` have no division, so the loop continues to `ensemble` or `bert_div` which do).
6. Division fill-in:
For projects where the winning section was supported only by `bert_base` and/or `tfidf` (neither `ensemble` nor `bert_div` voted for that section), no agreeing source has a division. In this case, the division is filled directly from `classifications_bert_division`, which has 100% division coverage.
7. Record: `{project_id, section, section_name, division, division_name, confidence, vote_count, total_voters, voters}`

5.3 Confidence Distribution

Confidence	Projects	Percentage	Meaning
very_high	23,058	43.4%	All 4 classifiers agreed
high	19,361	36.5%	Exactly 3 of 4 agreed
medium	9,871	18.6%	Exactly 2 of 4 agreed
low	790	1.5%	All 4 disagreed (LLM ti...

79.9% of all 53,080 projects were classified at high or very-high confidence.

6. Text Feature Enrichment

All three trained classifiers (bert_base, bert_div, tfidf) use an enriched text representation that goes beyond title + description:

```
def make_text(title, description, keywords=None, file_names=None):
    parts = []
    if title:      parts.append(title.strip())
    if description: parts.append(description.strip()[:600])
    if keywords:   parts.append(keywords.strip()[:200])
    if file_names:
        names = " ".join(
            f.rsplit(".", 1)[0].replace("_", " ").replace("-", " ")
            for f in file_names.split("|")[:10]
        )
        parts.append(names[:200])
    return " ".join(parts) or "untitled"
```

Feature	Projects with data	Coverage
title	~53,080	~100%
description	~52,000	~98%
keywords	~33,160	62%
file names (≤10)	~52,312	99%

File names are stripped of extensions and punctuation so that filenames like interview_transcripts_germany.xlsx become interview transcripts germany, providing topical signal without file-format noise.

7. Final Results

7.1 Coverage

Metric	Count	Percentage
Total projects	53,080	100%
With primary class (section)	53,080	100%
With secondary class (division)	53,080	100%

7.2 Section Distribution (All Repositories)

Section	Name	Projects	%
N	Professional, scientific...	~13,307	~25.1%
A	Agriculture, forestry a...	~8,779	~16.5%
R	Human health and socia...	~6,584	~12.4%

Section	Name	Projects	%
P	Public administration and social security	~6,499	~12.2%
Q	Education	~6,088	~11.5%
K	ICT activities	~1,581	~3.0%

Section	Name	Projects	%
S	Arts, sports and recreation	~1,462	~2.8%
(other 15 sections)	—	~8,780	~16.5%

Exact counts per repository are reported in the per-repository PDF reports.

7.3 Classifier Comparison

Classifier	Section Agreement	Division Agreement	Coverage
LLM ensemble	— (reference)	— (reference)	38% (qwen/mistral only)
bert_base	74.0%	77.6%	0%
bert_div	75.1%	40.9%	100%
tfidf	70.1%	69.6%	0%

The high agreement between bert_div and qwen (75.1%) but low agreement with claude (40.9%) reflects that bert_div was trained on qwen/mistral labels and inherits their subject-matter interpretation. bert_base shows more consistent agreement across both LLMs (74–77.6%), suggesting it learned more universal patterns.

8. Output Files

File	Description
23692652-sq26-classification.xlsx	53,080-row spreadsheet: repository_id, project_...
23692652-sq26-report-repo1.pdf	PDF report for Zenodo (39,405 projects): sectio...
23692652-sq26-report-repo10.pdf	PDF report for Harvard Dataverse (13,650 projec...
23692652-sq26-report-repo19.pdf	PDF report for Columbia University Libraries (2...
23692652-sq26.db	SQLite database with all raw classifier outputs...

Each PDF contains:

- Page 1: Bar histogram of all 22 ISIC sections with project counts
- Page 2: Rank-ordered table of the top-20 ISIC classes at the section+division level, plus auto-generated comments
- Page 3: Horizontal bar chart of the top-15 ISIC divisions, colour-coded by section

9. Database Schema (Key Tables)

```

projects
  id, repository_id, title, description, type, ...

classifications_combined      ← LLM results (Qwen2.5:7b, Mistral, Claude Haiku 4.5)
  project_id, model_name, section, section_name, division, division_name,
  confidence, method, files_used

classifications_bert_base     ← bert-base-uncased fine-tuned (section only)
  project_id, section, section_name, confidence, prob_top1, prob_top2

classifications_bert_division ← DistilBERT fine-tuned (section + division)
  project_id, section, section_name, division, division_name, confidence

classifications_tfidf         ← TF-IDF + Logistic Regression (section only)
  project_id, section, section_name, confidence, prob_top1, prob_top2

classifications_vote          ← FINAL: majority vote ensemble
  project_id, section, section_name, division, division_name,
  confidence, vote_count, total_voters, voters

classifications_files         ← Per-file BERT classification (QDA + QD only)
  project_id, project_type, file_name, section, section_name,
  confidence, prob_top1, prob_top2, section_top2

labels_training               ← Silver labels used to train bert_base, bert_div, tfidf
  project_id, section, section_name, source, confidence

```

10. Summary of Steps

Step	Script	Algorithm	Output Table	Division?
1	classify_isic.py /...	LLM prompt (Qwen2.5:7b, Mistral, Claude Haiku 4.5)	classifications_combined	Yes (qwen/mistral)
2	create_training_labels.py	Silver label extraction	labels_training	—
3	classify_bert.py	bert-base-uncased	classifications_bert_base	No
4	classify_bert_division.py	DistilBERT + PDF attention	classifications_bert_division	Yes (100%)
5	classify_tfidf.py	TF-IDF + Logistic Regression	classifications_tfidf	No
6	classify_vote.py	Majority vote with confidence	classifications_vote	Yes
7	Division fill-in (...)	Fill from bert_division	classifications_vote	100%
8	classify_files_bert.py	bert_base_isic_v1	classifications_files	No

11. File-Level Classification

Beyond project-level classification, each individual file belonging to a QDA_PROJECT or QD_PROJECT was also classified using the trained bert_base_isic_v1 model.

11.1 Data Source

File content was not fetched from Google Drive at classification time. Instead, the project_knowledge.file_snippets column stores a JSON dictionary {filename: text_snippet} that was populated during data ingestion. This column had 100% coverage across all 35,356 QDA and QD projects.

11.2 Text Construction per File

For each file, the input text was assembled as:

```
[parent project title] [parent description (first 400 chars)]
[parent keywords (first 150 chars)] [cleaned file name (first 150 chars)]
[file text snippet (first 300 chars)]
```

The file name was cleaned by stripping the extension and replacing underscores and hyphens with spaces. This gives the model both the broader research context (from the parent project) and the specific file signal.

11.3 Scale and Runtime

Project type	Files classified
QDA_PROJECT	13
QD_PROJECT	2,774
Total	2,787

Inference ran on Apple Silicon MPS at ~40 files/second. Total runtime: 1.2 minutes using batch size 64.

11.4 Results

Confidence	Files	%
very_high (≥80%)	1,886	67.7%
high (≥60%)	452	16.2%
medium (≥40%)	358	12.8%
low (<40%)	91	3.3%

File-level confidence (67.7% very_high) is notably higher than project-level confidence (43.4% very_high) because actual file text snippets provide stronger classification signal than metadata alone.

Top 5 ISIC sections across all classified files:

Section	Name	Files	%
A	Agriculture, forestry a.	527	18.9%
P	Public administration a	512	18.4%
E	Water supply; sewerag	426	15.3%
R	Human health and soci	257	9.2%
N	Professional, scientifi...	226	8.1%

Results are stored in the classifications_files table (script: classify_files_bert.py).

12. Classifier Role Assessment

Classifier	Role	Unique Contribution
LLM (ensemble)	Core — Essential	Highest reasoning quality; read...
bert_div	Core — Essential	Only trained classifier with 10...
bert_base	Supporting — Recommended	Strongest section classifier (7...
tfidf	Supporting — Optional	Lightweight orthogonal vote; in...

Minimum viable pipeline: LLM + bert_div covers 100% of projects at both classification levels. Adding bert_base raises the share of projects at high/very_high confidence from ~65% to ~80%.